

05

강

R 컴퓨팅

R 데이터의 구조 (2)

통계·데이터과학과 장영재 교수

학습목차

1 배열

2 리스트

3 데이터 프레임

4 실습하기

학습목표

- 1 배열의 특성을 이해한다.
- 2 리스트의 특성을 이해한다.
- 3 데이터 프레임의 특성을 이해한다.
- 4 배열, 리스트 및 데이터 프레임을 생성할 수 있다.
- 5 배열, 리스트 및 데이터 프레임과 관련한 연산을 실행할 수 있다.



01

배열

1. 배열

- ▶ 배열(array)은 행렬을 2차원 이상으로 확장시킨 객체를 의미
 - 2차원 구조로 이루어진 행렬도 일종의 배열이라고 할 수 있으며, 일반적으로는 3차원 이상으로 표현되는 객체

<배열의 주요 속성>

속성	설명
length	자료의 개수
mode	자료의 형태
dim	각 차원 벡터의 크기
dimnames	각 차원 리스트의 이름

1. 배열

예제 3-16 배열을 생성하는 데 사용되는 array() 함수를 이용하여 1에서 18까지의 원소를 갖는
3행 3열의 행렬 2개를 생성

```
> arr <- array(1:24, c(3,3,2))          # 1~18까지의 자료 생성

> dimnames(arr) <- list(paste("row", c(1:3)),paste("col",c(1:3)), paste("ar", c(1:2)))    #각 차원 이름을 지정

> arr
,, ar 1
col 1 col 2 col 3
row 1 1 4 7
row 2 2 5 8
row 3 3 6 9
,, ar 2
col 1 col 2 col 3
row 1 10 13 16
row 2 11 14 17
row 3 12 15 18
```

1. 배열

```
> length(arr)           # 자료의 개수 확인
[1] 18

> mode(arr)             # 자료의 형태 확인
[1] "numeric"

> dim(arr)               # 각 차원 벡터의 크기
[1] 3 3 2

> dimnames(arr)          # 각 차원 리스트의 이름
[[1]]
[1] "row 1" "row 2" "row 3"
[[2]]
[1] "col 1" "col 2" "col 3"
[[3]]
[1] "ar 1" "ar 2"
```

1. 배열

▶ 배열의 생성

- 배열은 행렬의 확장으로 기본적인 성질이 행렬과 유사
- 배열을 생성하는 방법
 - 1) array() 함수를 이용하여 직접 자료를 입력하는 방법
 - 2) dim() 함수를 이용하여 차원을 직접 지정

```
array(data, dim=c(행의 개수, 열의 개수, 행렬의 개수, ...), dimnames=NULL)
```

```
dim(x) <- c(행의 개수, 열의 개수, 행렬의 개수)
```

1. 배열

예제 3-17 ① array() 함수를 이용하여 1에서 6까지의 자료로 1차원 및 2차원 배열을 생성해 보고 1에서 8까지 값으로 3차원 배열을 만들기, ② dim() 함수를 이용하여 1~24까지 자료로 3행 4열 행렬 2개를 생성

```
> array(1:6)                # 1~6의 자료로 1차원 배열 생성
[1] 1 2 3 4 5 6

> array(1:6, c(2,3))        # 1~6의 자료로 2차원 배열 생성
[,1] [,2] [,3]
[1,] 1 3 5
[2,] 2 4 6

> array(1:8, c(2,2,2))      # 1~8의 자료로 3차원 배열 생성
,, 1
[,1] [,2]
[1,] 1 3
[2,] 2 4
,, 2
[,1] [,2]
[1,] 5 7
[2,] 6 8
```

1. 배열

```
arr <- c(1:24)           # 1~24의 자료 생성
> dim(arr) <- c(3,4,2)   # dim() 함수를 이용하여 3행 4열의 행렬 2개 생성
> arr
,, 1
[,1] [,2] [,3] [,4]
[1,] 1  4  7 10
[2,] 2  5  8 11
[3,] 3  6  9 12
,, 2
[,1] [,2] [,3] [,4]
[1,] 13 16 19 22
[2,] 14 17 20 23
[3,] 15 18 21 24
```

1. 배열

예제 3-18 array() 함수를 이용하여 생성된 배열 간 연산

> ary1 <- array(1:8, dim=c(2,2,2))	> ary2
> ary2 <- array(8:1, dim=c(2,2,2))	,, 1
> ary1	[1] [2]
,, 1	[1,] 8 6
[1] [2]	[2,] 7 5
[1,] 1 3	,, 2
[2,] 2 4	[1] [2]
,, 2	[1,] 4 2
[1] [2]	[2,] 3 1
[1,] 5 7	
[2,] 6 8	

1. 배열

예제 3-18 array() 함수를 이용하여 생성된 배열 간 연산

<code>> ary1 + ary2</code>	<code># 배열의 덧셈</code>	<code>> ary1 * ary2</code>	<code># 배열의 곱셈</code>
<code>,, 1</code>		<code>,, 1</code>	
<code>[1] [2]</code>		<code>[1] [2]</code>	
<code>[1,] 9 9</code>		<code>[1,] 8 18</code>	
<code>[2,] 9 9</code>		<code>[2,] 14 20</code>	
<code>,, 2</code>		<code>,, 2</code>	
<code>[1] [2]</code>		<code>[1] [2]</code>	
<code>[1,] 9 9</code>		<code>[1,] 20 14</code>	
<code>[2,] 9 9</code>		<code>[2,] 18 8</code>	

1. 배열

예제 3-18 array() 함수를 이용하여 생성된 배열 간 연산

> ary1 %*% ary2 # 두 배열	# 배열 원소의 추출/삭제
# 원소들의 곱의 합	[1] [2]
[1]	[1,] 1 3
[1,] 120	[2,] 2 4
> sum(ary1 * ary2)	> ary1[1,1,]
[1] 120	[1] 1 5
> ary1[,1]	> ary1[1,,2]
	[1] 1 3

02

리스트

2. 리스트

- ▶ 리스트는 서로 다른 형태(mode)의 데이터로 구성된 객체를 의미
 - 행렬과 배열 등이 동일한 형태의 원소로 이루어진 객체인 반면 리스트를 구성하는 성분(component)은 서로 다른 형태의 원소를 가질 수 있고, 길이도 다를 수 있음

<리스트의 주요 속성>

속성	설명
length	자료의 개수
mode	자료의 형태
names	각 구성요소의 이름

예제 3-19 list() 함수를 이용하여 2개의 벡터와 1개의 문자열을 가진 리스트를 생성하고 속성을 출력

```
> a <- 1:10
> b <- 11:15
> newlist <- list(a, b, "string") # 리스트를 생성
> length(newlist)                # 리스트 newlist의 자료의 개수
[1] 3
> mode(newlist)                  # 리스트 newlist의 자료의 형태
[1] "list"
```

2. 리스트

▶ 리스트의 생성 및 추출

- 리스트를 생성하고 추출하는 방법은 행렬·배열 등과 유사
- 특정한 성분을 추출하거나 성분 내의 특정 원소를 추출하려면 다음을 이용

- 1) `[ ]`를 사용하여 리스트에서의 성분을 추출
- 2) 성분의 이름이 부여되었다면 `$` 연산자로 구분됨
- 3) `[ ]` 연산자를 사용하여 성분 내의 특정 원소를 추출

2. 리스트

예제 3-20 list() 함수를 이용하여 "A"라는 문자와 1에서 8까지의 원소를 갖는 벡터를 성분으로 하는 list1이라는 리스트를 생성한 뒤, list1의 세 번째 성분으로 논리값 T, F를 갖는 벡터를 추가하고, 두 번째 성분의 아홉 번째 원소로 9를 추가

```
> list1 <- list("A", 1:8)           # list1 리스트 생성
> list1
[[1]]
[1] "A"
[[2]]
[1] 1 2 3 4 5 6 7 8
```

2. 리스트

예제 3-20 list() 함수를 이용하여 "A"라는 문자와 1에서 8까지의 원소를 갖는 벡터를 성분으로 하는 list1이라는 리스트를 생성한 뒤, list1의 세 번째 성분으로 논리값 T, F를 갖는 벡터를 추가하고, 두 번째 성분의 아홉 번째 원소로 9를 추가

```
> list1[[3]] <- c(T, F)      # 세 번째 성분을 추가
```

```
> list1[[2]][9] <- 9        # 두 번째 성분에 원소 추가
```

```
> list1
[[1]]
[1] "A"
[[2]]
[1] 1 2 3 4 5 6 7 8 9
[[3]]
[1] TRUE FALSE
> list1[[3]] <- NULL # 세 번째 성분 삭제
> list1[[2]] <- list1[[2]][-9] # 두 번째 성분의 아홉 번째 원소 삭제
> list1
[[1]]
[1] "A"
[[2]]
[1] 1 2 3 4 5 6 7 8
```

2. 리스트

예제 3-21 1에서 10까지의 원소를 갖는 벡터 a와 11에서 15까지의 원소를 갖는 벡터 b를

생성하고 "string"이라는 문자와 함께 newlist2라는 리스트를 생성

```
> a <- 1:10 ; b <- 11:15    # 벡터 a, b 생성
```

```
> newlist2 <- list(comp1=a, comp2=b, descrip="string")
```

```
> newlist2    # a, b 벡터 및 describe 변수에 string 문자 부여
```

```
$comp1
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
$comp2
```

```
[1] 11 12 13 14 15
```

```
$descrip
```

```
[1] "string"
```

```
> newlist2[[2]][5]          # 두 번째 성분 vec2의 다섯 번째 원소
```

```
[1] 15
```

```
> newlist2$comp2[c(2,3)]    # vec2의 두 번째와 세 번째 원소
```

```
[1] 12 13
```



데이터 프레임

3. 데이터 프레임

▶ 행렬은 차원으로 표시되며 같은 형태의 객체를 가지는 반면, 데이터 프레임은 각 열들이 서로 다른 형태의 객체를 가질 수 있음

▶ 데이터 프레임

<주요 특징>

- 1) 형태(mode)가 일반화된 행렬(matrix)
- 2) 하나의 객체에 여러 종류의 자료가 들어갈 수 있음
- 3) 각 열은 각각 변수와 대응
- 4) 분석이나 모형 설정에 적합한 자료 객체

3. 데이터 프레임

▶ 데이터 프레임의 생성

- 1) read.table() 함수를 이용하여 외부 텍스트 파일을 불러들이는 방법
- 2) data.frame() 함수를 이용하여 여러 종류의 자료 객체들을 서로 결합
- 3) as.data.frame() 함수를 사용하여 다른 형태의 자료 객체를 데이터 프레임의 형태로 변환

```
read.table(file, header=FALSE, sep="", ...)
```

```
data.frame(객체1, 객체2, ...)
```

```
as.data.frame(대상객체, ...)
```

예제 3-22 데이터가 다음과 같이 작성되어 “c:/data/ex1.txt”에 저장되어 있다고 가정할 때,

read.table() 함수를 이용하여 읽어 오기

```
num name age sex
1 Lee 55 M
2 Park 47 F
3 So 35 M
4 Kim 26 F
5 Yoon 29 M
```

<story.txt 데이터>

3. 데이터 프레임

```
> d2 <- read.table("c:/data/ex1.txt", row.names='num', header=T)
> d2
name age sex
1 Lee 55 M
2 Park 47 F
3 So 35 M
4 Kim 26 F
5 Yoon 29 M
```

- row.names='num'은 데이터의 num이라는 변수가 행 이름을 나타내며,
header=T는 첫 번째 줄이 변수명임을 의미

3. 데이터 프레임

예제 3-23 문자형 벡터 char1과 수치형 벡터 num1을 생성하고 결합하여 test1이라는 데이터 프레임을 생성

```
> char1 <- rep(LETTERS[1:3],c(2,2,1))          # 문자형 벡터 char1
```

```
> char1
```

```
[1] "A" "A" "B" "B" "C"
```

```
> num1 <- rep(1:3,c(2,2,1))                    # 수치형 벡터 num1
```

```
> num1
```

```
[1] 1 1 2 2 3
```

```
> test1 <- data.frame(char1, num1)             # test1 데이터 프레임 생성
```

```
> test1
```

```
char1 num1
```

```
1 A 1
```

```
2 A 1
```

```
3 B 2
```

```
4 B 2
```

```
5 C 3
```

3. 데이터 프레임

예제 3-24 문자 a부터 o까지로 이루어진 벡터를 생성하고 이를 5행 3열의 행렬로 변환한 뒤

a1으로 저장하고 데이터 프레임으로 변환

```
> a1 <- c("a","b","c","d","e","f","g","h","i","j","k","l","m","n","o")
```

```
> dim(a1) <- c(5,3)          # 5행 3열인 a1 행렬 생성
```

```
> a1
```

```
[,1] [,2] [,3]
```

```
[1,] "a" "f" "k"
```

```
[2,] "b" "g" "l"
```

```
[3,] "c" "h" "m"
```

```
[4,] "d" "i" "n"
```

```
[5,] "e" "j" "o"
```

3. 데이터 프레임

```
> test3 <- as.data.frame(a1)      # a1을 데이터 프레임으로 변환
```

```
> test3
```

```
V1 V2 V3
```

```
1 a f k
```

```
2 b g l
```

```
3 c h m
```

```
4 d i n
```

```
5 e j o
```

3. 데이터 프레임

▶ 데이터 프레임의 결합

- 서로 다른 데이터 프레임을 생성하였다면, 이 데이터 프레임을 합하여 하나의 데이터 프레임으로 생성할 수 있음
- 데이터 프레임을 합치는 방법
 - 1) cbind() 함수를 이용하여 옆으로 합치기
 - 2) rbind() 함수를 이용하여 아래로 합치기
 - 3) merge() 함수를 이용하여 병합하기

3. 데이터 프레임

예제 3-25 ① [예제 3-23]의 test1과 [예제 3-24]의 test3라는 데이터 프레임을 cbind() 함수를 이용하여 옆으로 병합,

② test4라는 새로운 데이터 프레임을 생성하고 rbind() 함수를 이용하여 test1과 test4를 아래로 병합

```
> cbind(test1,test3)
```

```
char1 num1 V1 V2 V3          # test1과 test3를 옆으로 합친 형태
```

```
1 A 1 a f k
```

```
2 A 1 b g l
```

```
3 B 2 c h m
```

```
4 B 2 d i n
```

```
5 C 3 e j o
```

```
> char1 <- rep(LETTERS[1:3],c(1,2,2))
```

```
> char1
```

```
[1] "A" "B" "B" "C" "C"
```

```
> num1 <- rep(1:3,c(1,1,3))
```

```
> num1
```

```
[1] 1 2 3 3 3
```

3. 데이터 프레임

```
> test4 <- data.frame(char1, num1)
```

```
> test4 # test4 데이터 프레임 생성
```

```
char1 num1
```

```
1 A 1
```

```
2 B 2
```

```
3 B 3
```

```
4 C 3
```

```
5 C 3
```

3. 데이터 프레임

```
> rbind(test1, test4)
```

```
char1 num1
```

```
# 아래로 합친 형태
```

```
1 A 1
```

```
2 A 1
```

```
3 B 2
```

```
4 B 2
```

```
5 C 3
```

```
6 A 1
```

```
7 B 2
```

```
8 B 3
```

```
9 C 3
```

```
10 C 3
```

3. 데이터 프레임

- merge() 함수를 사용하면 같은 변수에 대해서 한 번만 출력
- merge() 함수를 이용하여 test1과 test4를 결합할 경우에는 처음의 변수에 대해 알파벳 순서로 정렬되고 나머지 변수들이 옆으로 나열

```
> merge(test1, test4)
```

```
char1 num1          # test1, test4를 결합한 형태
```

```
1 A 1
```

```
2 A 1
```

```
3 B 2
```

```
4 B 2
```

```
5 C 3
```

```
6 C 3
```

04

실습하기

06강

다음시간안내 ▶▶▶

확장된 R데이터구조 (1)